

## Unit 1: Computational Thinking &amp; Algorithmic Logic • Topic 1

# Introduction to Programming & Computational Thinking, Building Blocks of Logic

Understanding what programming is, how programmers think, and the three logic building blocks of every program.

Every piece of software, from a calculator app to a self-driving car, starts as a set of instructions that a human designs and a computer follows. This topic introduces what programming really is, how programmers think about problems before writing any code (computational thinking), and the three basic 'building blocks' that every program, in every language, is built from.

## 1.1 What is a Program and What is Programming?

A **program** is a finite sequence of precise instructions, written in a language a computer can understand, that tells the computer exactly what steps to perform to solve a problem or complete a task. **Programming** (or coding) is the process of designing, writing, testing, and fixing (debugging) that sequence of instructions.

A computer is powerful but has no common sense — it does precisely what it is told, no more and no less. If the instructions are ambiguous, incomplete, or in the wrong order, the program will fail or produce the wrong result. This is why clear, logical thinking matters more than memorising a language's keywords.

## 1.2 From Human Language to Machine Language

Computers only understand **binary** (sequences of 0s and 1s), called **machine language**. Writing programs directly in binary is impractical, so programmers use **high-level languages** such as Python, which read almost like English and are then converted into machine language.

| Level               | Example           | Who understands it easily       |
|---------------------|-------------------|---------------------------------|
| Machine language    | 10110000 01100001 | The computer (directly)         |
| Assembly language   | MOV AL, 61h       | Hardware-aware programmers      |
| High-level language | print("Hello")    | Humans — close to English/Maths |

Two common ways of converting high-level code into machine code are a **compiler** (translates the whole program at once, before running it — e.g. C, C++) and an **interpreter** (translates and executes the program line-by-line, as it runs — e.g. Python). Python is an *interpreted, high-level, general-purpose* language, which is why Python programs are quick to write and test but usually run slower than compiled languages.

## 1.3 What is Computational Thinking?

**Computational thinking** is a problem-solving process that programmers use before writing a single line of code. It is not about 'thinking like a computer' — it is about breaking a problem down so clearly that a computer (which has no common sense) can be told exactly what to do. It rests on four pillars:

- **Decomposition** – breaking a large, complex problem into smaller, more manageable sub-problems. Example: 'build a calculator app' becomes 'read two numbers' + 'read an operator' + 'perform the operation' + 'display the result'.

- **Pattern Recognition** – spotting similarities or trends between and within problems, so a solution used once can be reused. Example: noticing that finding the largest of 2 numbers and the largest of 10 numbers uses the same repeated ‘compare and keep the bigger one’ idea.
- **Abstraction** – focusing on the essential details and ignoring the irrelevant ones. Example: a ‘calculate average marks’ program does not need to know a student’s name or address.
- **Algorithm Design** – developing the actual step-by-step solution (the algorithm) that can be translated into code.

**REAL-LIFE ANALOGY:** Computational thinking is a life-skill, not just a coding skill. Planning a birthday party by breaking it into tasks (guest list, venue, food, invites) is decomposition in everyday life.

## 1.4 Building Blocks of Logic

However complex a program looks, its logic is built from only **three control structures**. Every algorithm and every Python program is some combination of these three:

### (a) Sequence

Instructions are executed **one after another**, in the exact order they are written, from top to bottom. This is the default flow of any program.

```
name = input("Enter your name: ")    # Step 1
print("Hello,", name)                # Step 2 (runs after Step 1)
print("Welcome to Python!")          # Step 3 (runs after Step 2)
```

### (b) Selection (Decision)

The program **chooses** between two or more paths depending on whether a condition is True or False. This is implemented using if, if-else and if-elif-else statements (covered in detail in Unit 2).

```
age = 20
if age >= 18:
    print("Eligible to vote")
else:
    print("Not eligible to vote")
```

### (c) Iteration (Repetition / Looping)

A set of instructions is **repeated** multiple times, either a fixed number of times or until a condition becomes False. This is implemented using for and while loops (covered in detail in Unit 2).

```
for i in range(1, 4):
    print("Iteration number", i)
# Output:
# Iteration number 1
# Iteration number 2
# Iteration number 3
```

**Any program — from a simple ‘Hello World’ to a large operating system — is built purely by combining and nesting these three building blocks: Sequence → Selection → Iteration.**

## 1.5 Syntax vs Logic (Two Different Kinds of Correctness)

| Aspect    | Meaning                                                | Example of an error                                         |
|-----------|--------------------------------------------------------|-------------------------------------------------------------|
| Syntax    | The grammar rules of the language                      | Missing colon: <code>if x &gt; 5 print(x)</code>            |
| Logic     | Whether the steps actually solve the problem correctly | Using <code>+</code> instead of <code>*</code> to find area |
| Semantics | The intended meaning behind the instruction            | Program runs, but gives wrong output                        |

A program can be free of syntax errors (it runs without crashing) yet still have a **logic error** that produces the wrong answer. Computational thinking mainly protects against logic errors; the interpreter catches syntax errors for you.

## 1.6 Key Terms Recap

- **Algorithm** – a well-defined, ordered sequence of steps to solve a problem (details in Topic 2).
- **Source code** – the human-readable program text written by a programmer.
- **Interpreter** – a program (like the Python interpreter) that reads and executes source code directly, line by line.
- **Debugging** – the process of finding and fixing errors (bugs) in a program.
- **IDE** – an Integrated Development Environment (e.g. VS Code, PyCharm, Jupyter, IDLE) used to write, run and debug code conveniently.

## Practice Questions

### A. Multiple Choice Questions

- Which of the following best defines a 'program'?  
(A) A single instruction given to hardware (B) A finite sequence of precise instructions to solve a task (C) A type of computer hardware (D) An error message
- Python is best described as a:  
(A) Compiled, low-level language (B) Interpreted, high-level language (C) Assembly language (D) Machine language
- Which pillar of computational thinking means breaking a big problem into smaller ones?  
(A) Abstraction (B) Pattern Recognition (C) Decomposition (D) Algorithm Design
- Which of these is NOT one of the three building blocks of logic?  
(A) Sequence (B) Selection (C) Compilation (D) Iteration
- A program that runs without crashing but produces the wrong answer most likely has a:  
(A) Syntax error (B) Logic error (C) Hardware error (D) Compilation error

### B. True / False

- A compiler translates and executes a program line by line as it runs.
- Ignoring irrelevant details of a problem and focusing only on what matters is called abstraction.
- Every Python program is built using only sequence, selection and iteration in some combination.
- Machine language is easier for humans to read than Python.

### C. Short Answer Questions

- Differentiate between a compiler and an interpreter with one example of a language that uses each.
- Explain, with a real-life (non-computing) example, what decomposition means.
- What is the difference between a syntax error and a logic error? Give one example of each.
- List the three building blocks of logic and briefly describe each in one line.

### D. Long Answer / Descriptive Questions

- Explain computational thinking and its four pillars (decomposition, pattern recognition, abstraction, algorithm design) with a suitable example for each, using the problem 'planning a class trip'.
- Describe, with examples, how sequence, selection and iteration combine to form the logic of a real program (e.g. a program that prints whether numbers from 1 to 10 are even or odd).

### E. Programming Problems

- Identify which building block(s) — sequence, selection, iteration — would be needed to write a program that asks a user for 5 marks, one at a time, and prints PASS/FAIL for each based on a passing mark of 40. (No code required — just explain your reasoning.)
- Write, in plain English (not Python), an algorithm outline that uses decomposition to break the problem 'prepare a cup of tea' into at least 5 smaller steps.

**Answer Key (A & B):** MCQ: 1-B, 2-B, 3-C, 4-C, 5-B | T/F: 6-False, 7-True, 8-True, 9-False